Report 2: Requirements Specification

Personal Budget Tracker for Students

Team Members:
Zeki Metalla (Team Leader) — zmetalla23@epoka.edu.al
Pamela Ajas— pajas23@epoka.edu.al
Merjeme Bega— mbega23@epoka.edu.al
Xhulio Mullaj— xmullaj23@epoka.edu.al

Epoka University — Department of Computer Engineering

1.Chosen Development Model

We choose agile because it allows flexibility and continuous improvement during development. Since student needs may change (adding new expense categories or features), Agile helps adapt quickly. It also supports incremental development, testing features step by step.

2. User Requirements

The following user requirements describe the needs of the two main actors in the system: students (primary users) and administrators. Each requirement is written as a user story. All listed requirements are implemented in the current version of the application.

ID	User Story
UR-01	As a student, I want to register an account so that I can start tracking my personal finances.
UR-02	As a student, I want to log in with my email and password so that I can securely access my financial data.
UR-03	As a student, I want to add income and expense transactions so that I can keep a record of all my financial activities.
UR-04	As a student, I want to categorize my transactions (food, transport, books, etc.) so that I can see where my money goes.
UR-05	As a student, I want to set a monthly budget so that I can control my overall spending.
UR-06	As a student, I want to receive alerts when I approach or exceed my budget so that I can adjust my spending.
UR-07	As a student, I want to view monthly reports with charts so that I can understand my spending patterns.
UR-08	As a student, I want to plan travel budgets based on destination countries so that I can estimate how much a trip will cost.
UR-09	As a student, I want to record charity donations so that I can track my charitable contributions.
UR-10	As a student, I want to plan my weekly routine with associated costs so that I can align my budget with my daily schedule.
UR-11	As a student, I want to switch between dark and light themes so that I can use the application comfortably.
UR-12	As an admin, I want to manage all student accounts so that I can activate, deactivate, or remove users.
UR-13	As an admin, I want to view system-wide financial reports so that I can monitor overall platform usage.
UR-14	As an admin, I want to manage default categories and country cost data so that students have accurate information.

3. Functional Requirements

The following functional requirements define the specific behaviors and functions of the system. They are grouped by module and all are fully implemented in the current version. The system contains two main modules: Student Panel and Admin Panel.

ID	Module	Requirement
FR-01	Authentication	The system shall allow students to register with full name, email, and password.
FR-02	Authentication	The system shall authenticate users via email and password with bcrypt-hashed password verification.
FR-03	Authentication	The system shall redirect users to the appropriate dashboard based on their role (admin or student) after login.
FR-04	Authentication	The system shall terminate the user session and redirect to the login page upon logout.
FR-05	Transactions	The system shall allow students to add income transactions with category, amount, description, and date.
FR-06	Transactions	The system shall allow students to add expense transactions with category, amount, description, and date.
FR-07	Transactions	The system shall allow students to delete their own transactions.
FR-08	Transactions	The system shall allow filtering transactions by type (income/expense), month, and category.
FR-09	Budget	The system shall allow students to set a total monthly budget amount.
FR-10	Budget	The system shall allow students to set spending limits for individual expense categories.
FR-11	Budget	The system shall calculate and display the percentage of budget used in the current month.
FR-12	Budget	The system shall generate automatic notifications when spending reaches 80% or exceeds 100% of the monthly budget.
FR-13	Budget	The system shall calculate and display the recommended daily spending based on remaining budget and days left.
FR-14	Reports	The system shall generate a 6-month income vs. expense trend bar chart on the dashboard.
FR-15	Reports	The system shall generate a doughnut chart showing expense distribution by category.
FR-16	Reports	The system shall display daily spending as a line chart for any selected month.
FR-17	Reports	The system shall display a yearly expense overview as a bar chart.
FR-18	Reports	The system shall list the top 5 highest expenses for a selected month.
FR-19	Travel	The system shall allow students to create travel plans by selecting a destination country, dates, and number of people.
FR-20	Travel	The system shall automatically estimate travel costs (flight, hotel, food, transport, activities) based on stored country cost data.
FR-21	Travel	The system shall allow students to update actual travel expenses and compare them against estimates.
FR-22	Charity	The system shall allow students to record donations with organization name, amount, date, and recurring status.

FR-23	Charity	The system shall display a 6-month donation trend chart.
FR-24	Routine	The system shall allow students to create weekly routine activities with title, day, time range, category, and estimated cost.
FR-25	Routine	The system shall calculate and display the total weekly and monthly routine costs.
FR-26	Categories	The system shall provide default income and expense categories with icons and colors.
FR-27	Categories	The system shall allow students to create custom categories with custom icon and color.
FR-28	Admin	The system shall allow admins to view, add, activate, deactivate, and delete student accounts.
FR-29	Admin	The system shall allow admins to reset student passwords.
FR-30	Admin	The system shall allow admins to view all transactions and donations across the platform.
FR-31	Admin	The system shall allow admins to manage default categories and country travel cost data.
FR-32	Admin	The system shall log all admin actions (user creation, deletion, settings changes) in an activity log.
FR-33	Admin	The system shall provide system-wide reports with per-student financial breakdowns.
FR-34	Profile	The system shall allow users to update their full name, email, and currency preference.
FR-35	Profile	The system shall allow users to change their password with current password verification.

4. Non-Functional Requirements

The following non-functional requirements define the quality attributes, constraints, and standards of the system. They are grouped by type: Security, Performance, Usability, Compatibility, Maintainability, Reliability, and Scalability. All are implemented in the current version.

ID	Type	Requirement
NFR-01	Security	The system shall hash all passwords using bcrypt before storing them in the database.
NFR-02	Security	The system shall use PDO prepared statements for all database queries to prevent SQL injection.
NFR-03	Security	The system shall sanitize all user inputs with htmlspecialchars() to prevent XSS attacks.
NFR-04	Security	The system shall enforce role-based access control, preventing students from accessing admin pages and vice versa.
NFR-05	Security	The system shall protect sensitive directories (config, sql, includes) via .htaccess rules.
NFR-06	Security	The system shall set session cookies as HTTP-only to prevent client-side script access.
NFR-07	Performance	The system shall load dashboard data including charts within 3 seconds on a standard connection.
NFR-08	Performance	The system shall use output buffering (ob_start) to prevent header conflicts and improve response handling.
NFR-09	Performance	The system shall limit query results (e.g., recent transactions to 8, notifications to 10) to avoid unnecessary data loading.
NFR-10	Usability	The system shall provide a responsive design that works on desktop, tablet, and mobile devices.
NFR-11	Usability	The system shall support dark and light theme switching with preference persistence via localStorage.
NFR-12	Usability	The system shall display flash messages for all user actions (success, error) with auto-dismiss after 5 seconds.
NFR-13	Usability	The system shall provide confirmation dialogs before all delete operations.
NFR-14	Usability	The system shall display a sidebar navigation with clear section grouping (Main Menu, Planning, Settings).
NFR-15	Usability	The system shall animate stat cards on page load for improved visual feedback.
NFR-16	Compatibility	The system shall use Bootstrap 5.3.2 for cross-browser CSS compatibility.
NFR-17	Compatibility	The system shall use Chart.js 4.4.0 for rendering all charts and graphs.
NFR-18	Compatibility	The system shall use the Inter font from Google Fonts for consistent typography across platforms.
NFR-19	Compatibility	The system shall use CDN-hosted libraries (Bootstrap, Chart.js, Bootstrap Icons) requiring no local dependency installation.
NFR-20	Maintainability	The system shall separate concerns using includes (header.php, footer.php) for consistent layout across pages.

NFR-21	Maintainability	The system shall use a centralized configuration file (config/database.php) for all database and application settings.
NFR-22	Maintainability	The system shall organize admin and student modules in separate directory structures.
NFR-23	Maintainability	The system shall use CSS custom properties (variables) for theming, enabling easy color scheme changes.
NFR-24	Reliability	The system shall prevent duplicate budget alert notifications by checking if the same alert was already created today.
NFR-25	Reliability	The system shall prevent deletion of categories that have associated transactions.
NFR-26	Reliability	The system shall use foreign key constraints with CASCADE delete to maintain data integrity.
NFR-27	Reliability	The system shall validate all form inputs server-side before database operations.
NFR-28	Scalability	The system shall use a relational database schema with proper indexing (unique keys, foreign keys) for efficient queries.
NFR-29	Scalability	The system shall store country cost data in a dedicated table allowing easy addition of new countries by admin.
NFR-30	Scalability	The system shall support multiple currencies (ALL, EUR, USD) configurable per user.

5. Acceptance Criteria for Functional Requirements

The following acceptance criteria are defined for 5 selected functional requirements, using the Given-When-Then format to describe testable conditions.

5.1 FR-02 — User Authentication via Email and Password

AC #	Acceptance Criterion
AC-1	Given a registered student with valid credentials, when they enter their email and password and click Sign In, then the system authenticates them and redirects to the student dashboard.
AC-2	Given a registered admin with valid credentials, when they enter their email and password and click Sign In, then the system authenticates them and redirects to the admin dashboard.
AC-3	Given an invalid email or password, when the user attempts to log in, then the system displays the error message 'Invalid email or password.' and does not create a session.
AC-4	Given a deactivated student account, when the user attempts to log in with correct credentials, then the system displays 'Your account has been deactivated. Contact the administrator.' and denies access.
AC-5	Given a successful login, when the session is created, then the system records the last_login timestamp in the database.

5.2 FR-05 — Add Income Transactions

AC #	Acceptance Criterion
AC-1	Given a logged-in student on the Transactions page, when they click 'Add Transaction', select type 'Income', choose a category, enter an amount > 0, enter a date, and click Save, then the transaction is stored in the database with the correct user_id, type, category_id, amount, and date.
AC-2	Given the transaction form, when the user selects 'Income' type, then only income categories are displayed in the category dropdown.
AC-3	Given a successful transaction creation, then the system displays a flash message 'Transaction added successfully!' and the transaction appears in the list.
AC-4	Given missing required fields (amount = 0, no date, or no category), when the user clicks Save, then the system displays 'Please fill in all required fields.' and does not save.

5.3 FR-12 — Automatic Budget Alert Notifications

AC #	Acceptance Criterion
AC-1	Given a student whose monthly expenses reach 80% of their set monthly budget, when any page with the header loads, then the system creates a warning notification with title 'Budget Warning' showing the percentage used and remaining amount.
AC-2	Given a student whose monthly expenses reach or exceed 100% of their monthly budget, when any page loads, then the system creates a danger notification with title 'Budget Exceeded!' showing the spent and budgeted amounts.
AC-3	Given that the same alert has already been created today, when the budget check runs again, then no duplicate notification is created.
AC-4	Given a student with no monthly budget set (budget = 0), then no budget alerts are generated regardless of spending.

5.4 FR-20 — Automatic Travel Cost Estimation

AC #	Acceptance Criterion
AC-1	Given a student creating a travel plan with a destination country that exists in the country_costs table, when the plan is submitted, then the system automatically generates 6 expense line items: Flight, Accommodation, Food & Meals, Local Transport, Activities, and Miscellaneous.
AC-2	Given the travel plan parameters (country, number of days, number of people), then Accommodation cost = avg_hotel_per_night × days × ceil(people/2), Food cost = avg_meal_cost × 3 meals × days × people, Transport cost = avg_transport_daily × days × people.
AC-3	Given a successfully created travel plan, then the total_budget field is updated with the sum of all estimated expense line items.
AC-4	Given the travel detail view, when the student enters an actual cost for an expense item and clicks the check button, then the actual_cost field is updated in the database.

5.5 FR-28 — Admin User Management

AC #	Acceptance Criterion
AC-1	Given a logged-in admin on the Manage Users page, then all student accounts are listed with their name, email, status, transaction count, total spent, last login, and join date.
AC-2	Given the admin clicks the deactivate button for an active student, then the student's is_active field is set to 0 and the action is logged in admin_logs.
AC-3	Given the admin clicks the activate button for an inactive student, then the student's is_active field is set to 1 and the action is logged.
AC-4	Given the admin clicks delete for a student and confirms, then the student and all associated data (transactions, donations, routines, travel plans, notifications) are permanently removed via CASCADE delete.
AC-5	Given the admin resets a student's password with a new password of at least 6 characters, then the password is hashed with bcrypt and updated in the database.

6. Acceptance Criteria for Non-Functional Requirements

The following acceptance criteria are defined for 5 selected non-functional requirements, verifying that the system meets its quality and constraint standards.

6.1 NFR-01 — Password Hashing with Bcrypt

AC #	Acceptance Criterion
AC-1	Given a new user registration, when the password is stored, then it is hashed using PHP's password_hash() with PASSWORD_DEFAULT (bcrypt) algorithm.
AC-2	Given a login attempt, when the password is verified, then the system uses password_verify() against the stored hash — plain text passwords are never compared directly.
AC-3	Given any database query or admin view, then raw passwords are never visible or retrievable from the database.
AC-4	Given a password change operation, then the new password is re-hashed before storage.

6.2 NFR-04 — Role-Based Access Control

AC #	Acceptance Criterion
AC-1	Given a user with role 'student', when they attempt to access any URL under /admin/, then the system redirects them to dashboard.php.
AC-2	Given a user with role 'admin', when they access the admin panel, then all admin pages are accessible.
AC-3	Given an unauthenticated user (no session), when they attempt to access any protected page, then the system redirects to login.php.
AC-4	Given the login process, when authentication succeeds, then the user's role is stored in $_SESSION['user_role'] and checked on every protected page via requireAdmin() or requireStudent() functions.

6.3 NFR-10 — Responsive Design

AC #	Acceptance Criterion
AC-1	Given a screen width below 992px (tablet/mobile), then the sidebar is hidden by default and accessible via a hamburger menu toggle button.
AC-2	Given a mobile device, when the sidebar is open, then a dark overlay appears and clicking it closes the sidebar.
AC-3	Given any screen size, then all tables use the table-responsive wrapper class enabling horizontal scroll on narrow screens.
AC-4	Given Bootstrap 5.3.2 grid system, then all card layouts use responsive column classes (col-sm-6, col-xl-3, col-lg-8, etc.) adapting to screen width.

6.4 NFR-20 — Separation of Concerns via Includes

AC #	Acceptance Criterion
AC-1	Given the student module, then every page includes header.php (sidebar, navbar, session check) and footer.php (scripts, closing tags) from the includes/ directory.

AC-2	Given the admin module, then every page includes admin-specific header.php and footer.php from admin/includes/ with the admin sidebar navigation.
AC-3	Given the configuration, then all database connection logic, helper functions, and authentication checks are centralized in config/database.php.
AC-4	Given any page, then no database credentials or connection code appears outside config/database.php.

6.5 NFR-26 — Data Integrity via Foreign Key Constraints

AC #	Acceptance Criterion
AC-1	Given a user deletion, then all associated records in transactions, donations, travel_plans, routines, budget_limits, and notifications tables are automatically deleted via ON DELETE CASCADE.
AC-2	Given a travel plan deletion, then all associated travel_expenses are automatically removed via CASCADE.
AC-3	Given an attempt to delete a category that has transactions, then the system displays an error and prevents the deletion.
AC-4	Given the database schema, then all foreign key relationships are defined with appropriate ON DELETE actions (CASCADE or SET NULL).